Ingeniería de Software II

Persistencia

¿Qué es la persistencia?

La persistencia es la característica de los datos, que sobreviven al programa que los genera.

Es necesario porque la memoria es **volátil** y **acotada**.

¿Qué es la persistencia?

A tener en cuenta al seleccionar un mecanismo de persistencia:

- Performance
  - Acceso al medio (memoria, disco)
  - Busquedas
- Transformación de datos
- Tolerancia a fallos

Tipos de Persistencia

Veremos los siguientes:

- Archivos
- Prevalencia (patrón de diseño)
- Bases de datos
  - Relacional
  - Objetos

Archivos

- Planos
  - Ancho fijo
  - Ancho variable (e.g. CSV)
- Estructurado (e.g. XML)
- Archivos binarios
  - Archivos indexados
  - Serialización

Logs

Cada **registro** (fila lógica) ocupa una cantidad fija de caracteres, y dentro de ese registro, **cada campo** también tiene un tamaño predefinido

```
0001Juan Perez      28Argentina
0002Maria Lopez     31Chile
0003Carlos Gomez    45Uruguay
```

+ JSON

```
[
  {
    "id": 1,
    "nombre": "Juan Perez",
    "edad": 28,
    "pais": "Argentina"
  },
  {
    "id": 2,
    "nombre": "Maria Lopez",
    "edad": 31,
    "pais": "Chile"
  }
]
```

Intercambio de datos

Rendimiento

Comunicación

Archivos binarios

Formato no directamente legible por humanos ➜ Para ser leído por un programa
Ejemplos de archivos binarios: .jpg, .mp3, .mp4, .dat, .sqlite...

- Archivos indexados
    1. Permite acceder rápidamente a ciertos registros usando una clave o índice
    2. Ventaja: búsqueda rápida.
    3. Desventaja: hay que mantener actualizado el índice cuando se cambian los registros
- Serialización
    1. Convertir una estructura de datos (por ej, un objeto) en una secuencia de bytes o texto para poder guardarla en un archivo o enviarla por red
    2. Ejemplo de archivo binario serializable: formato Pickle de Python

Prevalencia



Los datos se mantiene en memoria en formato nativo.

Las transacciones se registran, y se hacen *snapshots* a disco.

Implementaciones:

- Prevayler (Java)
- Madeleine (Ruby)
- Bamboo (.net)

¿Redis? Más o menos
❌ Prevalencia pura = se persiste el estado de objetos
❌Redis = se persisten estructuras de datos clave-valor
✅ Implementa el mismo principio arquitectónico

Prevalencia

- Ventajas
  - No hay transformación, los datos se guardan en el formato que los usa el programa.
  - Alto grado de transparencia.
  - La performance es muchísimo más alta.
- Desventajas
  - La aplicación debe poder ser contenida en memoria.
  - Interoperabilidad.

Los snapshots suelen ser serializaciones binarias del estado interno de los objetos (por ejemplo, usando ObjectOutputStream en Java o pickle en Python).

Bases de Datos

- Independencia de la forma en que se guardan físicamente los datos
- Permite compartir la DB entre varios programas
- Mecanismos para recuperarse si algo falla
- Permite establecer reglas de integridad
- Performance y escalabilidad
- Puedo ejecutar cosas dentro de la base

Bases de Datos Relacionales

Organiza datos en tablas (relaciones de filas y columnas)

Basada en teoría de álgebra relacional y conjuntos

Posee tablas, constraints, keys, etc.

Se debe normalizar.

Caso típico: DWH
- Los datos se extraen de sistemas transaccionales
- Se transforman y agregan.
- Se almacenan en estructuras multidimensionales optimizadas para lectura.

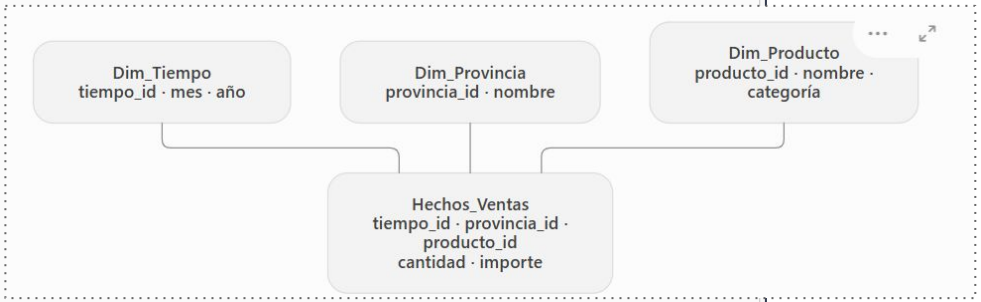
Bases de Datos Multidimensionales

Se almacenan grandes volúmenes históricos

Se guarda desnormalizado (para no hacer joins)

Se precalculan posibles agregaciones

Se utilizan para reportes solamente

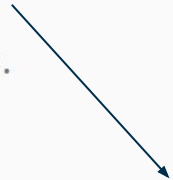


Bases de Datos de Objetos

Guarda objetos complejos (no simples tablas)

Posee relaciones del paradigma de objetos (e.g. herencia)

Soporta relaciones de muchos a muchos.



Ejemplos:
- ZODB: Python
- Objectivity/DB: C++, Java, Python
- db4o: Java y .NET

- En OO hay herencia, encapsulamiento y referencias.
- En SQL hay tablas, claves y joins.
- No hay necesidad de hacer mapeos ni usar un ORM (como Hibernate o SQLAlchemy).

Mapeo Objeto-Relacional (ORM)

Son frameworks que mapean objetos (existen en el software) a tablas relacionales (existen en la base de datos).

Automatizan las operaciones más comunes (CRUD).

Facilitan la programación, a costa de performance, y complejidad.

Bases de datos NO-SQL

- Surgen en parte para suplir limitaciones del modelo relacional
  - Escalabilidad
  - Distribuidas “por diseño”
  - Volumen de datos
  - Esquemas flexibles
  - Pueden o no implementar ACID
- Algunos tipos
  - Columnas
  - Key/value
  - Documentos

A - Atomicidad (Atomicity): Una **transacción es una unidad indivisible** de trabajo
Ejemplo: Si transferís $100 de una cuenta A a una cuenta B, ambas operaciones (restar de A y sumar a B) deben completarse.

C - Consistencia (Consistency): Una **transacción lleva la base de datos de un estado válido a otro estado válido**
Ejemplo: Si hay una regla que dice "el saldo no puede ser negativo", ninguna transacción puede violar esta restricción.

I - Aislamiento (Isolation): Las **transacciones concurrentes se ejecutan de forma aislada**, como si fueran **secuenciales**.
Ejemplo: Si dos personas intentan comprar el último producto disponible simultáneamente, solo una transacción debe tener éxito.

D - Durabilidad (Durability): Una vez que una **transacción se confirma** (commit), los **cambios son permanentes**, incluso ante fallos del sistema (caídas de energía, crashes).
Ejemplo: Después de confirmar un pago, ese registro persiste aunque el servidor se reinicie inmediatamente después.

NO-SQL - Columnas

Se usa principalmente en **análisis de grandes volúmenes de datos** (Big Data, BI, IoT, métricas, logs).

- Los datos son guardados ordenados por columna en lugar de por registro.
  - Las búsquedas son mucho más rápidas
  - Mas fácil calcular proyecciones relativas a una o pocas columnas.
  - Es más difícil realizar escrituras o consultas de muchas columnas.
- Ejemplos:
  - BigTable (Google)
  - Hypertable

Apache Cassandra



NO-SQL - Columnas

Row Oriented
(RDBMS Model)

id	Name	Age	Interests
1	Ricky		Soccer, Movies, Baseball
2	Ankur	20	
3	Sam	25	Music

Multi-valued

null

Column Oriented
(Multi-value sorted map)

id	Name
1	Ricky
2	Ankur
3	Sam

id	Age
2	20
3	25

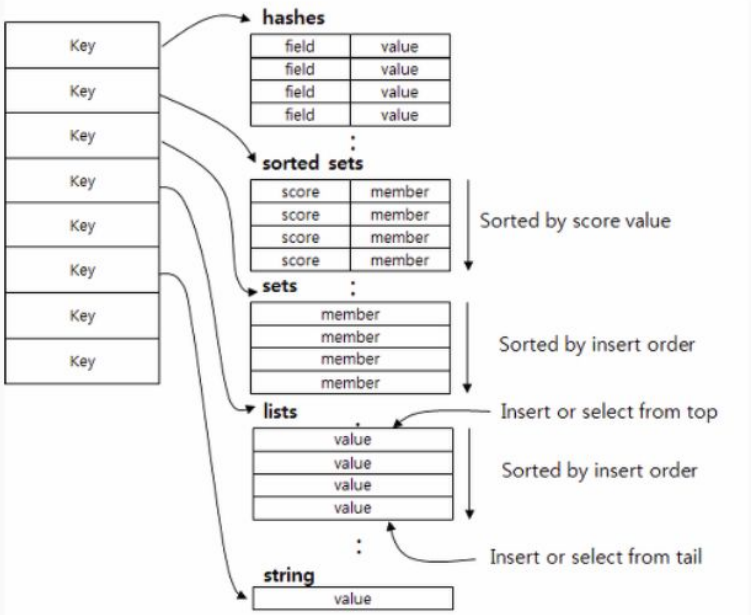
id	Interests
1	Soccer
1	Movies
1	Baseball
3	Music

NO-SQL - Clave/Valor

- La información se guarda organizado como un par clave/valor.
  - Tiene una alta flexibilidad, no es necesario modificar estructuras para agregar atributos.
  - No es necesario lidiar con tipos de datos
  - Es complicado hacer búsquedas por más de un campo (AND y OR)

- Ejemplos:
  - Redis
  - Dynamo

NO-SQL - Clave/Valor



NO-SQL - Documentos

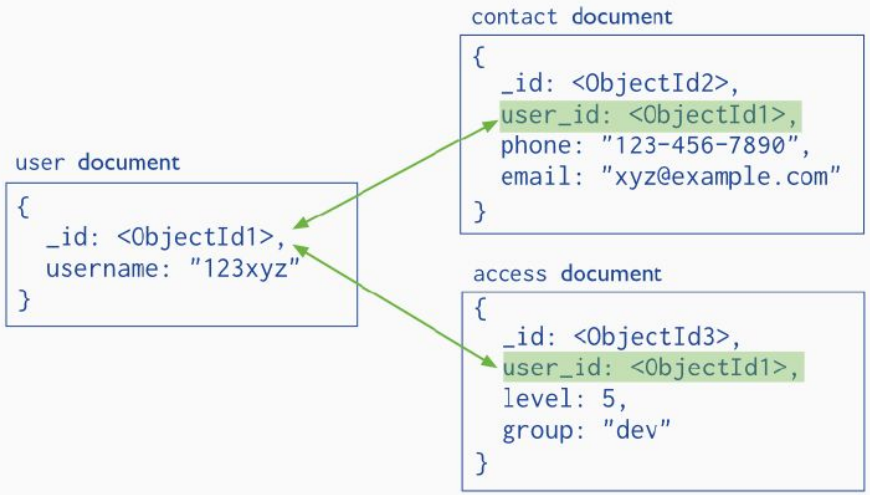
- Guardan cualquier “documento” arbitrario.
  - Es mas fácil guardar estructuras que varían.
  - Suelen usar XML, YAML o JSON para retornar los datos a través de un protocolo REST.
  - No es tan fácil hacer consultas complejas (proyecciones, por ejemplo)



elegir qué columnas de una tabla querés obtener

- Ejemplos:
  - MongoDB (sourceforge, github)
  - CouchDB

NO-SQL - Documentos



Bases de Datos Vectoriales

Las bases de datos vectoriales almacenan y gestionan **vectores multidimensionales** (**embeddings** → representaciones numéricas de texto, imágenes, audio u otros datos generadas por modelos de IA)

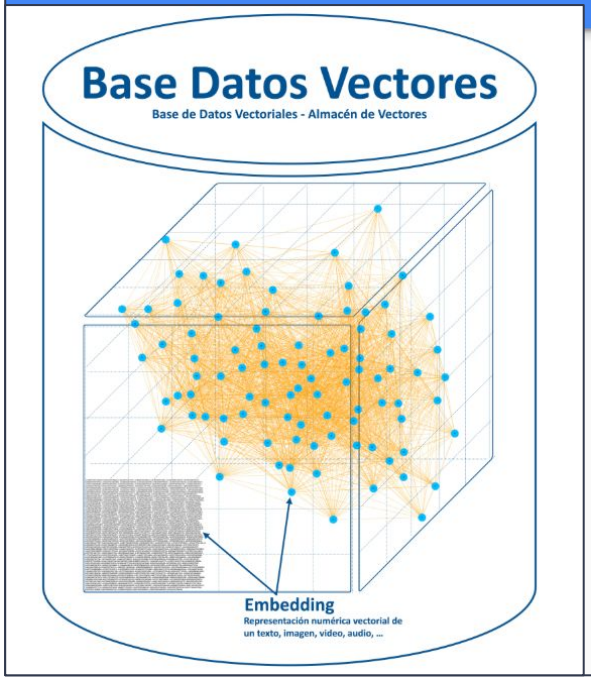
Su objetivo principal es **buscar similitudes semánticas entre elementos**, no coincidencias exactas.

Ejemplos: Pinecone, Qdrant, Chroma

Usos:
- No reemplazan a las relacionales, se integran con ellas para ofrecer **búsquedas semánticas**
- Fundamentales en sistemas modernos de **IA generativa**.

Funcionamiento:
- Codificación: un modelo (ej. BERT, OpenAI Embeddings) transforma el dato en un vector.
- Almacenamiento: los vectores se guardan junto con metadatos.
- Consulta: al buscar, el sistema calcula la distancia (coseno, euclidiana, etc.) entre el vector de consulta y los almacenados.
- Resultado: devuelve los elementos más similares semánticamente.

Bases de Datos Vectoriales



Qué hace la base de datos vectorial:

- La base de datos vectorial es como el mapa 3D de palabras (para un texto)
- Guarda las posiciones (los “vectores”) de cada palabra.
- Busca cuáles están más cerca de la que preguntás.
- Devuelve las más parecidas.

Etapa	Época	Hito principal
Fundamentos teóricos	1970–2000	Búsqueda por similitud y estructuras espaciales
Embeddings y deep learning	2013–2017	Word2Vec, FAISS, Annoy
Bases vectoriales dedicadas	2019–2021	Milvus, Weaviate, Pinecone, Qdrant
Expansión masiva (IA generativa)	2022–hoy	Integración con LLMs y RAG, boom del ecosistema

Persistencia - Resumen

1. Prevalencia: guarda todo el estado del sistema en memoria y lo persiste periódicamente mediante snapshots o logs de transacciones.
- Ejemplo: implementaciones caseras de in-memory persistence.
- Casos típicos: sistemas financieros o de simulación con datos complejos que deben responder rápido (baja latencia, pocos usuarios).
- No recomendado: el conjunto de datos excede la memoria disponible, hay muchos usuarios concurrentes o se necesita consulta ad hoc (no está optimizada para búsquedas).

2. Archivos: almacenamiento en archivos planos o estructurados (CSV, JSON, XML, binarios, etc.).
- Ejemplo: logs de auditoría, configuración, resultados de procesos batch.
- Casos típicos: intercambio simple de datos, backups, persistencia mínima en apps de escritorio o scripts.
- No recomendado: aplicaciones con concurrencia, integridad referencial o necesidad de consultas complejas. Falta atomicidad y consistencia transaccional.

3. Bases de datos relacionales: almacenamiento estructurado en tablas con relaciones (modelo entidad–relación) y lenguaje SQL.
- Ejemplo: PostgreSQL, MySQL, SQL Server.
- Casos típicos: sistemas transaccionales, ERP, CRM, sistemas bancarios.
- No recomendado: datos no estructurados, grandes volúmenes de datos distribuidos o alta variabilidad de esquema.

Persistencia - Resumen

4. Bases de datos multidimensionales: organizan los datos en cubos con dimensiones y medidas para análisis rápido (OLAP).
- Ejemplo: Microsoft Analysis Services, Apache Kylin.
- Casos típicos: inteligencia de negocios (BI), dashboards, análisis histórico, reportes agregados.
- No recomendado: transacciones online (OLTP), actualizaciones frecuentes o escritura intensiva.

5. Bases de datos orientadas a objetos (OODB): almacenan objetos completos con su estructura y relaciones, sin convertirlos a tablas.
- Ejemplo: db4o, ObjectDB, Versant.
- Casos típicos: aplicaciones que manejan jerarquías y grafos complejos.
- No recomendado: integración con sistemas externos o consultas SQL; baja adopción y soporte limitado.

6. Bases de datos NoSQL por columnas: los datos se almacenan por columnas, no por filas; optimizadas para lectura analítica y compresión.
- Ejemplo: Apache Cassandra, HBase.
- Casos típicos: grandes volúmenes de datos distribuidos, telemetría, analítica en tiempo real.
- No recomendado: transacciones fuertes o estructuras relacionales complejas.

Persistencia - Resumen

7. Bases de datos NoSQL por clave/valor: los datos se almacenan como pares {clave: valor}, sin estructura fija.
- Ejemplo: Redis, Amazon DynamoDB.
- Casos típicos: caché, sesiones de usuario, colas de mensajes.
- No recomendado: búsquedas por atributos o relaciones; solo acceden por clave.

8. Bases de datos NoSQL por documentos: almacenan documentos semiestructurados (JSON, XML). Cada documento contiene todo lo necesario.
- Ejemplo: MongoDB, CouchDB, Firestore.
- Casos típicos: catálogos de productos, APIs, apps móviles.
- No recomendado: cuando se requiere consistencia transaccional fuerte o relaciones complejas.

9. Bases de datos vectoriales: almacenan embeddings o representaciones numéricas de datos (texto, imágenes, audio) para búsquedas por similitud.
- Ejemplo: Pinecone, FAISS.
- Casos típicos: búsqueda semántica, recomendadores, RAG (Retrieval-Augmented Generation), IA generativa.
- No recomendado: datos estructurados, operaciones transaccionales o reportes agregados.